



SENAI

Herança em Python

Conceito

A herança é similar ao conceito de herança no mundo real. Uma criança herda alguns traços de seus pais ao mesmo tempo em que apresenta características próprias.

Em programação orientada a objetos esse conceito é similar em alguns aspectos.



Pode existir herança entre classes diferentes, e essa relação é do tipo “é um”. Por exemplo, um “laptop” é um produto, um “carro” é um veículo, e um “empregado” é uma pessoa. Aqui, “laptop” é uma classe filha de produto, “carro” é filha de veículo, e “empregado” é filha de pessoa.

1. **Reuso do código**
2. **Manutenção Simplificada**
3. **Organização do Código**
4. **Polimorfismo***
5. **Abstração***

Sintaxe da classes

```
class Produto: #Classe pai ou superclasse
```

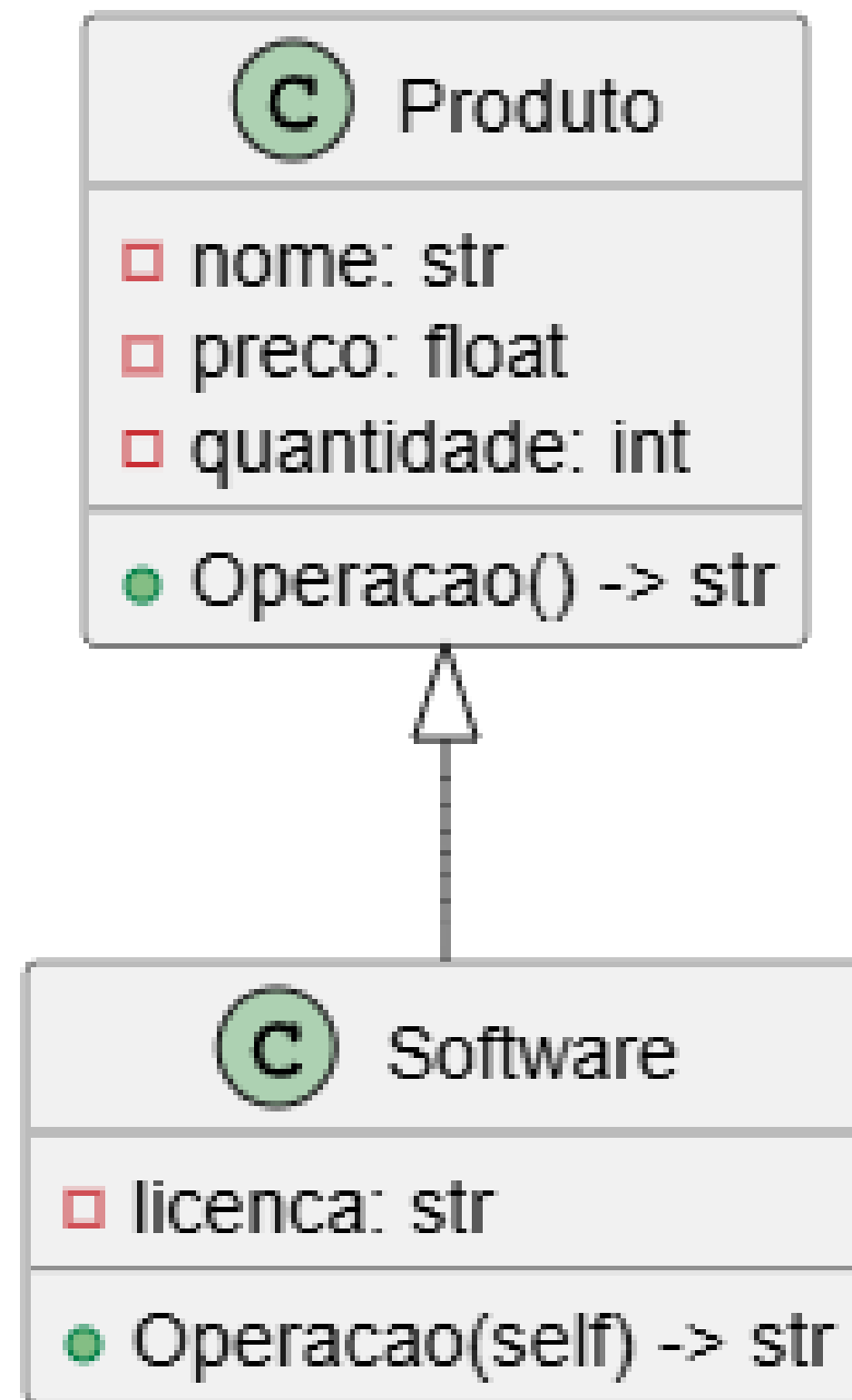
```
class software (Produto): #Classe filha ou subclasse
```


Sintaxe do construtor das classes

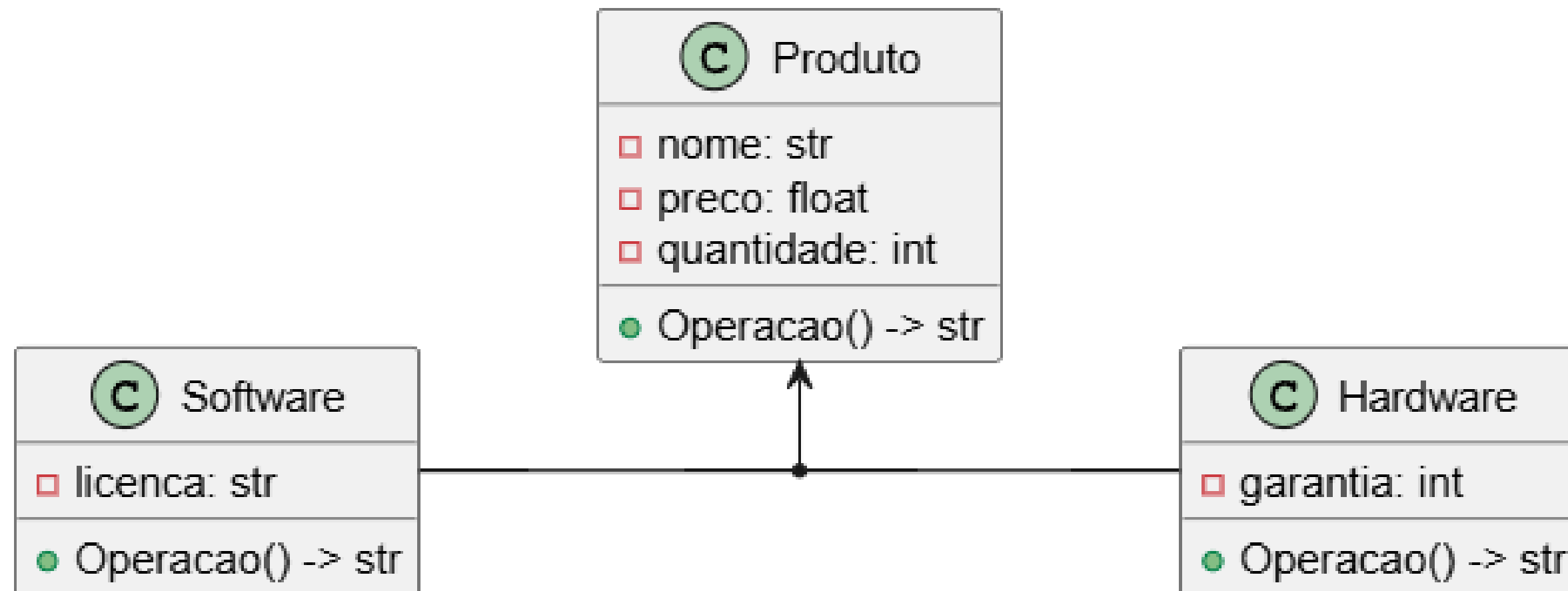
```
class Produto: #Classe pai ou superclasse
    ...
    # Construtor
    def __init__(self, nome: str, preco: float, quantidade: int):
        self.__nome = nome
        self.__preco = preco
        self.__quantidade = quantidade
```

```
class software (Produto): #Classe filha ou subclasse
    ...
    # Construtor
    def __init__(self, nome: str, preco: float, quantidade: int, licenca: str):
        super().__init__(nome, preco, quantidade)
        self.__licenca = licenca
```

Projeto UML das classes



Projeto UML das classes



Problema exemplo 1 Herança

Suponha um negócio de banco que possui uma conta comum e uma conta para empresas, sendo que a conta para empresa possui todos membros da conta comum, mais um limite de empréstimo e uma operação de realizar empréstimo.

C ContaPessoaFisica

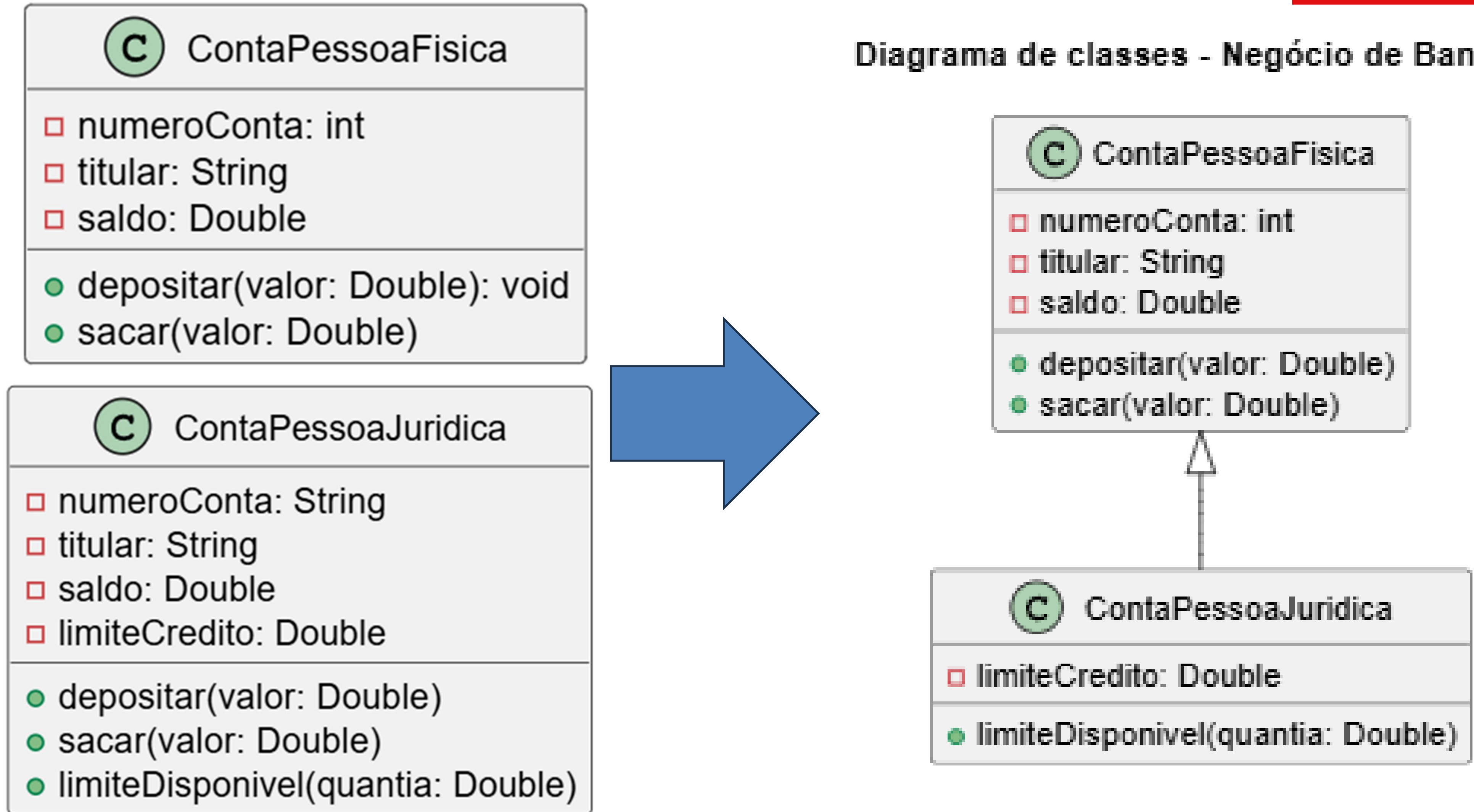
- ❑ numeroConta: int
- ❑ titular: String
- ❑ saldo: Double
- depositar(valor: Double): void
- sacar(valor: Double)

C ContaPessoaJuridica

- ❑ numeroConta: String
- ❑ titular: String
- ❑ saldo: Double
- ❑ limiteCredito: Double
- depositar(valor: Double)
- sacar(valor: Double)
- limiteDisponivel(quantia: Double)

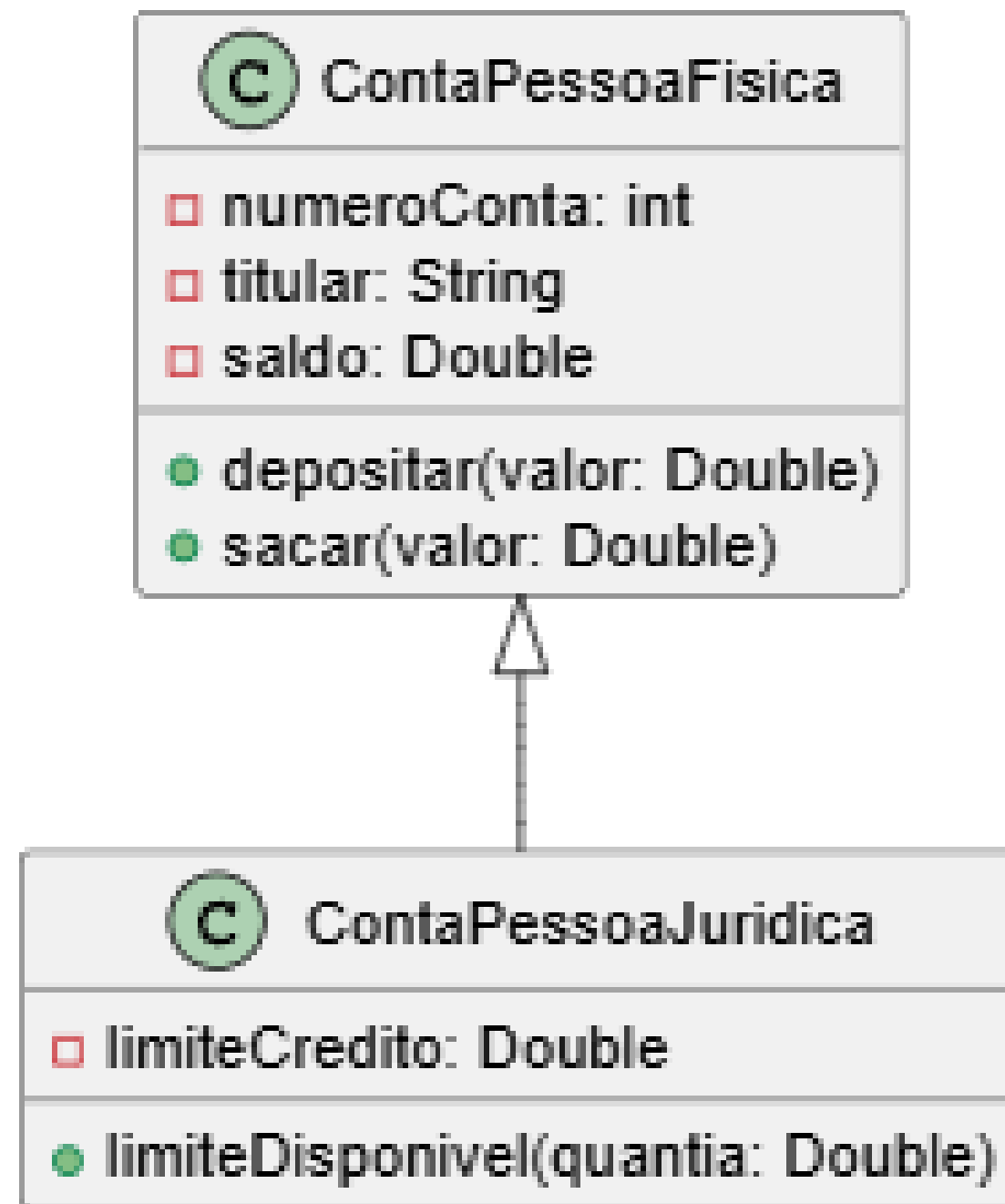
Resolvendo com herança

Diagrama de classes - Negócio de Banco



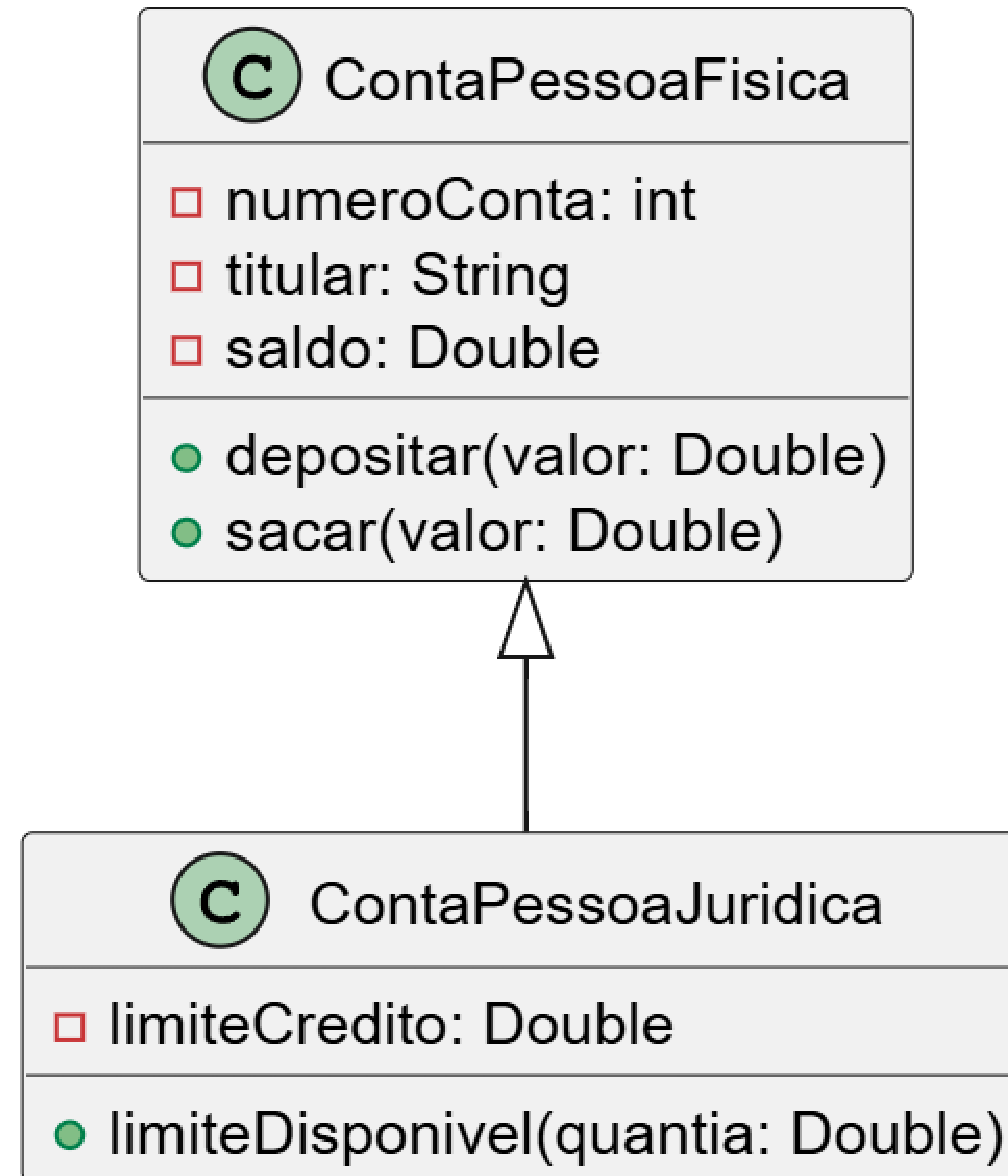
Definições importantes

Diagrama de classes - Negócio de Banco



- Relação "é-um";
- Generalização/especialização;
- Superclasse (classe base) / subclasse (classe derivada)
- Herança / extensão;
- Herança é uma associação entre classes (e não entre objetos)

Diagrama de classes - Negócio de Banco



Sobreposição

Sobreposição em python

Em Python, não existem as palavras-chave **override**, **virtual** ou **base** para controle de sobreposição de métodos, pois a sobreposição é implícita e ocorre automaticamente quando uma classe derivada define um método com o mesmo nome e assinatura de um método na classe base. O comportamento é determinado pelo objeto em tempo de execução, que invoca a implementação da classe específica (derivada ou base).

Sobreposição em python - exemplo

```
class ClasseBase:
    def metodo(self):
        print("Executando o método da classe base.")

class ClasseDerivada(ClasseBase):
    def metodo(self): # Método com o mesmo nome e assinatura da classe base
        print("Executando o método da classe derivada (sobrescrito).")

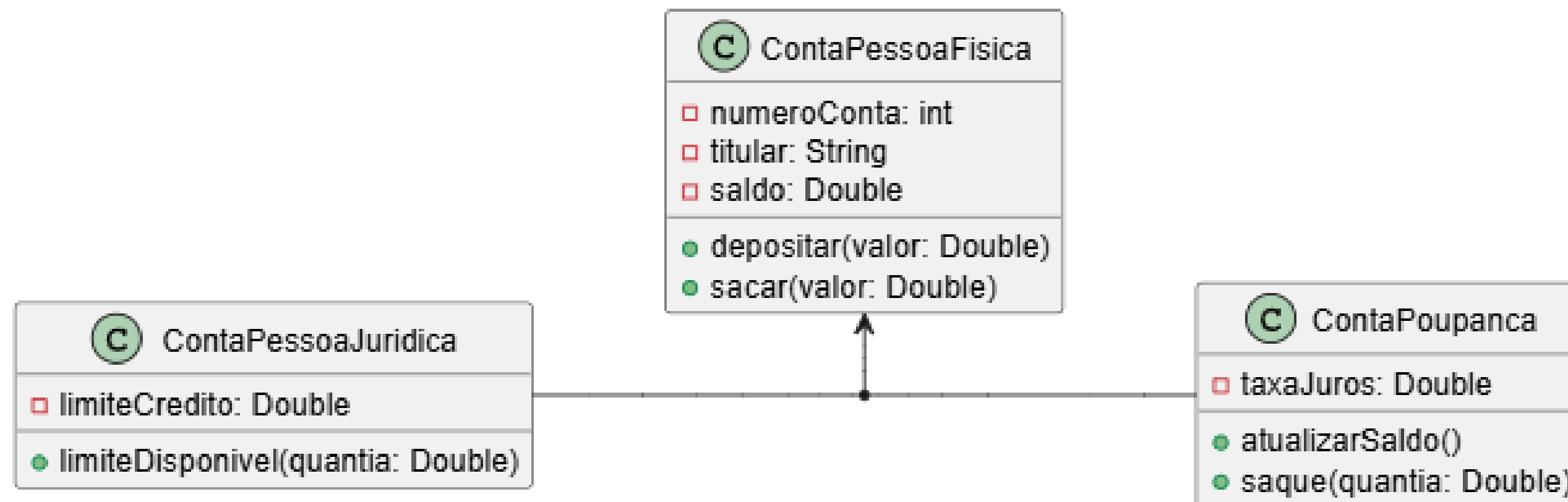
# Instanciando a classe derivada
obj_derivada = ClasseDerivada()
obj_derivada.metodo() # Saída: Executando o método da classe derivada
(sobrescrito).

# Instanciando a classe base
obj_base = ClasseBase()
obj_base.metodo() # Saída: Executando o método da classe base.
```

Exemplo 2 - Herança

Suponha as seguintes regras para saque:

Diagrama de classes - Negócio de Banco



- **Conta comum:**

para saques é cobrada uma taxa no valor de 5.00.

- **Conta poupança:**

não é cobrada taxa.

Exemplo 2 - Herança

Suponha as seguintes regras para saque:

```
Depósito de R$500.00 realizado com sucesso.  
Saque de R$200.00 realizado com sucesso.  
Saldo PF: R$1300.00  
Limite disponível para 4000? True  
Saldo atualizado com juros de 2.00%. Juros aplicados: R$30.00  
Saque de R$300.00 realizado com sucesso.  
Saldo Poupança: R$1230.00
```

- **Conta comum:**

para saques é cobrada
uma taxa no valor de 5.00.

- **Conta poupança:**

não é cobrada taxa.

Classes

e

métodos selados

Evita que a classe **Seja** herdada.

Classes e métodos selados

Python não tem suporte nativo para o conceito de "classes seladas" ou "métodos selados" como em outras linguagens.

Em vez disso, para impedir que uma classe seja herdada, você pode utilizar a subclasse de `object` e garantir que nenhum método ou atributo possa ser sobrescrito, ou, como uma alternativa mais eficaz, usar o decorador `@final` do módulo `typing` (disponível a partir do Python 3.8) para tornar as classes e métodos explicitamente imunes à herança e sobrescrita, respectivamente.

Mas para que?

Segurança:

Dependendo das regras do negócio, às vezes é desejável garantir que uma classe não seja herdada, ou que um método não seja sobreposto. Geralmente convém selar métodos sobrepostos, pois sobreposições múltiplas podem ser uma porta de entrada para inconsistências

Mas para que?

Performance:

Atributos de tipo de uma classe selada são analisados de forma mais rápida em tempo de execução.

- Exemplo clássico: string

Exemplo de aplicação

```
from typing import final

@final
class Forma(object): # Classe selada (não pode ser herdada)
    def __init__(self, cor):
        self.cor = cor

    @final
    def calcular_area(self): # Método selado (não pode ser sobrescrito)
        return "Área não especificada"

class Circulo(Forma):
    def __init__(self, cor, raio):
        super().__init__(cor)
        self.raio = raio

    def calcular_area(self): # Erro de tempo de compilação ou runtime, dependendo do contexto
        return f"Área do círculo: {3.14 * self.raio ** 2}"

# Tentativas de herança e sobrescrita (que falhariam ou dariam erro)
# c = Circulo("vermelho", 5) # A definição de Circulo falharia por ser @final
# c.calcular_area() # A chamada também falharia por ser um método @final
```

Exercícios



Google Classroom



A nighttime photograph of a city street, likely in São Paulo, featuring light trails from moving vehicles and illuminated buildings in the background. The image is used as a background for a promotional graphic.

SENAI

DEPARTAMENTO REGIONAL
DE SÃO PAULO

www.sp.senai.br